

Sketch 1: NBA Live Game Win Probability & Momentum Visualizer

Project Description: This project is a real-time data dashboard designed for basketball fans and sports analytics enthusiasts to track the live NBA games. The app visualizes live win probability as a continuous line graph that fluctuates with every basket, turnover, foul, or timeout. Instead of just showing a static percentage number, this tool lets users interactively scroll back through the game timeline. Users can filter the view to isolate specific quarters or overlay historical comeback data from similar past games to see how rare the current live scenario actually is.

Use Case Vignette: Jared is watching a live playoff game between the Celtics and the Bucks. The Bucks are down by 15 points in the 3rd quarter, but suddenly start making a comeback. Jared opens the web app on his laptop, selects the live game, and watches the win probability graph shift in real time. Intrigued by a massive 20% spike in the Celtics' favor right before halftime, he hovers his mouse over that specific dip in the timeline graph. A contextual window pops up, showing a description or play-by-play breakdown: *"3:12 remaining - 3-pointer by Tatum, followed by a Bucks turnover."* Jared uses a slider to overlay a historical plot of the 2021 Finals, comparing this exact scenario to a famous past comeback.

- **Users & Stakeholders:** Primary users are sports fans, fantasy basketball players, and amateur sports journalists. Secondary stakeholders could include live sports bettors looking for analytical context.
- **The Task & Problem Solved:** Standard sports apps only show box scores or rigid play-by-play text lists. This solves the problem of visualizing the abstract waves of a basketball game with historical context and real-time data.
- **Primary Interaction & Loop:** Selecting a live or archived game from a list. The primary loop involves scrubbing across a chronological timeline graph, adjusting filters, and clicking peaks/valleys to update the play-by-play detail panel.
- **User Interface Speculation:** I would build it as a web app, and look into different framework options. The core layout would feature a plotly graph representing the win probability line.
- **Data Input, Sources & Processing:** Data is fed from a sports API like the Balldontlie API or rapidAPI NBA. The Python backend pulls the live score, time remaining, possession, and current stat line to calculate the win probability every 10 seconds.
- **Results Presentation:** Presented as a live-updating line chart split down the middle with Team A on top and Team B on bottom. Momentum shifts are color-coded, and summary statistic tables are underneath the main graph.

- Implementation Challenges: Handling real-time API polling and state updates without hitting API rate limits or having lag.
- Self-Implementation Assessment: I can easily handle the charts and drawing the data from the API. Handling the real-time data would be a bit more difficult as well as testing it with live examples since the NBA season is coming to an end.

Sketch 2: Playlist Generator

Project Description: This web app allows users to describe an aesthetic then parses this to automatically generate a personalized 10-song Spotify playlist. Unlike generic playlist generators, this can use the Spotify API to look at the user's listening history and favorite genres. It combines the text's mood with the user's personal tastes, so the recommendations feel relevant. Users can add and remove songs to customize the playlist to their liking.

Use Case Vignette: Maya wants some music to study to but is tired of her usual lo-fi beats. She opens the app and types: *"Vintage 1970s vinyl lounge, warm lighting, and old paperback books."* Because Maya listens to a lot of indie rock, the app references her top artists and blends them with the warm and vintage text parameters to generate a custom 10-song playlist featuring tracks from artists like Fleetwood Mac and Tame Impala. Maya clicks "Save to Spotify," and the playlist is instantly pushed to her phone.

- Users & Stakeholders: Creative professionals, students looking for focused background environments, and everyday music fans.
- The Task & Problem Solved: Traditional music search requires knowing artist names or rigid genres. This solves choice paralysis by turning abstract, emotional language directly into a curated listening experience.
- Primary Interaction & Loop: The primary interaction is plain text entry in a large prompt box. Once generated, the user can enter and interactive loop by changing their prompt and resubmitting before saving the playlist.
- User Interface Speculation: The interface would be a highly visual but minimalist frontend. The page dynamically divides into clean blocks of the prompt and an embedded Spotify player widget.
- Data Input, Sources & Processing: Text input can be processed using an LLM API to extract emotional keywords. These keywords map to acoustic targets like danceability or energy. The backend uses Python Spotify library to pull the user's top tracks and generate recommendations.

- Results Presentation: Displayed as a clean digital canvas and a live playable Spotify playlist widget embedded directly.
- Implementation Challenges: Securely configuring authentication for individual users to access their Spotify data and translating text into exact Spotify track attributes.
- Self-Implementation Assessment: I can design the frontend aesthetic but will probably struggle with setting up the Spotify user authentication.

Sketch 3: Localized Budget Grocery & Personalized Meal Planner

Project Description: This project acts as an assistant designed to help users optimize their weekly meals without overspending. Users input their specific dietary preferences, allergies, and a weekly budget. The system compares these against localized grocery store pricing to generate a fully customized 7-day meal plan that minimizes cost while dietary targets. The main interface features a calendar layout alongside a shopping list. The app compares prices across multiple local chains to show where the user can save the most money.

Use Case Vignette: Michael is a student managing a \$50 weekly grocery budget. He opens the app, inputs a preference for high-protein meals, flags a tree nut allergy, and locks the budget slider at \$50. The app builds a Monday-through-Sunday meal grid. Instead of recommending generic, expensive health foods, it schedules meals around budget-friendly items currently on sale. When he reviews the shopping list, a comparison table shows that buying the ingredients at Aldi will cost \$36.42, whereas a competing store would cost \$47.10. He can save the plan and export it as a checklist.

- Users & Stakeholders: Primary users are students, low-to-middle-income families, and people into fitness tracking macros on a budget. Secondary stakeholders include grocery store managers seeking to highlight weekly specials or nutrition educators.
- The Task & Problem Solved: Traditional meal planning is exhausting and getting harder as grocery prices keep rising. This solves choice paralysis and financial strain by automating the cross-referencing of health constraints directly with shifting grocery store prices.
- Primary Interaction & Loop: A step-by-step for dietary and budget configuration. The primary user interaction loop involves reviewing the generated calendar, clicking an individual meal to swap it if it's unappealing, and watching the total budget tracker and shopping list dynamically recalculate.
- User Interface Speculation: The frontend uses a clean layout featuring a weekly calendar grid, interactive slide for budget ceiling, and recipes from the ingredient list.

- Data Input, Sources & Processing: The backend utilizes a recipe database from Spoonacular API. Pricing profiles would be simulated using mock data representing weekly local grocery flyers or open consumer price index trends.
- Results Presentation: Results will be displayed as an interactive weekly meal calendar, a shopping list, and a comparison of competing stores.
- Implementation Challenges: The biggest challenge will be sourcing live pricing data.
- Self-Implementation Assessment: I can use the API to find recipes within dietary restrictions and making sure the shopping list is within budget, but I will struggle with real-time pricing data.